The following code does is not compliant with SOLID.

This code defines two classes: `Item` and `ShoppingCart`. The `Item` class represents an item with attributes like name, weight, and price. The `ShoppingCart` class takes a list of items and calculates the total price of the items in the cart and to estimate shipping costs based on weight and distance. The example usage creates a shopping cart with a book and a shirt, then prints the total price, standard shipping cost, and express shipping cost for a distance of 100 units.

```python
class Item:
    def __init__(self, name, weight, price):
        self.name = name
        self.weight = weight
        self.price = price

    def __str__(self):
        return f"{self.name} (weight: {self.weight:.2f}, price: ${self.price:.2f})"


class ShoppingCart:
    def __init__(self, items):
        self.items = items
        self.weight = sum(item.weight for item in items)

    def get_total_price(self):
        return sum(item.price for item in self.items)

    def get_standard_shipping_cost(self, distance):
        base_cost = 5
        weight_cost = 2 * self.weight
        distance_cost = 0.1 * distance
        return base_cost + weight_cost + distance_cost

    def get_express_shipping_cost(self, distance):
        base_cost = 10
        weight_cost = 3 * self.weight
        distance_cost = 0.2 * distance
        return base_cost + weight_cost + distance_cost


# Example usage
cart = ShoppingCart([Item("Book", 1, 10), Item("Shirt", 0.5, 20)])
print(f"Total Price: ${cart.get_total_price()}")
print(f"Standard Shipping Cost: ${cart.get_standard_shipping_cost(100)}")
print(f"Express Shipping Cost: ${cart.get_express_shipping_cost(100)}")
```

Provided code violates several SOLID principles, which are:

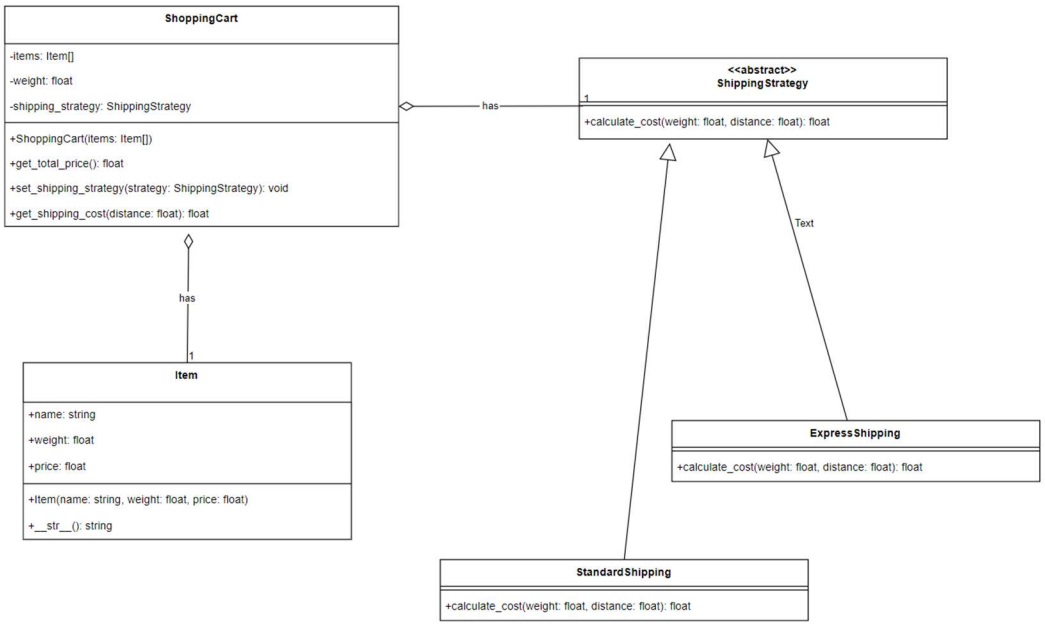
1. Single Responsibility Principle (SRP): The ShoppingCart class is responsible for several things: managing items, calculating total price, and calculating shipping

costs for different types (standard and express). This violates the SRP as the class has multiple reasons to change.

2. Open/Closed Principle (OCP): Adding new shipping options requires modifying the ShoppingCart class to add new methods (get_next_day_shipping_cost, etc.). This violates the OCP, which suggests code should be open for extension (adding new options) but closed for modification (of existing functionalities).

3. Liskov Substitution Principle (LSP): While not directly violated here, there's potential for violation if the code is extended further. Imagine introducing a "Free Shipping" option that doesn't have a base cost or weight/distance-based calculations. It wouldn't be a true substitute for other shipping strategies that rely on these calculations, violating the LSP.

Here's how you can improve the code to adhere to SOLID principles:



1. Introduce an interface for shipping strategies: Define an interface `ShippingStrategy` with a method `calculate_cost(weight, distance)`.

2. Create concrete classes for different shipping strategies: Implement `StandardShipping`, `ExpressShipping`, and potentially others, inheriting from `ShippingStrategy` and providing their specific cost calculation logic.

3. Refactor the ShoppingCart class: Inject the desired `ShippingStrategy` object into the `ShoppingCart` constructor or through a setter method. Modify the `get_shipping_cost(self, distance)` method to call the `calculate_cost` method on the injected strategy object, receiving the cost based on the chosen strategy.

Reconstruct the following code snippets to make it work. Write the code in the box on the left. You may repeat the snippet more than once.

Step 1 - Define the `ShippingStrategy` interface:

The `ShippingStrategy` class will have two subclasses: `StandardShipping` and `ExpressShipping`. Each subclass will implement the `calculate_cost` method to calculate the shipping cost based on the weight and distance.

```
class ShippingStrategy(ABC):
  @abstractmethod
  def calculate_cost(self, weight, dist
ance):
    pass


class StandardShipping(ShippingStrategy
):
  def calculate_cost(self, weight, dist
ance):
    base_cost = 5
    weight_cost = 2 * weight
    distance_cost = 0.1 * distance
    return base_cost + weight_cost + di
stance_cost


class ExpressShipping(ShippingStrategy)
:
  def calculate_cost(self, weight, dist
ance):
    base_cost = 10
    weight_cost = 3 * weight
    distance_cost = 0.2 * distance
    return base_cost + weight_cost + di
stance_cost
```

```
    base_cost = 5
    weight_cost = 2 * weight
    distance_cost = 0.1 * distance
```

```
    pass
```

```
  def calculate_cost(self, weight, d
istance):
```

```
class StandardShipping(ShippingStrat
egy):
```

```
    return base_cost + weight_cost +
distance_cost
```

```
class ExpressShipping(ShippingStrate
gy):
```

```
  @abstractmethod
```

```
    base_cost = 10
    weight_cost = 3 * weight
    distance_cost = 0.2 * distance
```

```
class ShippingStrategy(ABC):
```

Step 2 - Refactor the ShoppingCart class:

```
class ShoppingCart:
  def __init__(self, items):
    self.items = items
    self.weight = sum(item.weight for item
in self.items)

  def get_total_price(self):
    return sum(item.price for item in self.
items)

  def set_shipping_strategy(self, strategy)
:
    self.shipping_strategy = strategy

  def get_shipping_cost(self, distance):
    return self.shipping_strategy.calculate
_cost(self.weight, distance)
```

```
self.shipping_strategy = strategy
```

```
    return self.shipping_strategy.ca
lculate_cost(self.weight, distance)
```